ENTERPRISE SAAS ENGINEERING PLAYBOOK

NexusCloud SaaS:

From CTO to Junior Engineer: The Complete Software Lifecycle with DDD & Agile Scrum

---

From Business Inception to Live Support • Cross-Functional Squads • Team Topologies
Java 21 • Spring Boot 3 • Spring Cloud • PostgreSQL 16 • Redis 7
Elasticsearch 8 • Apache Kafka • Next.js 15 • GCP GKE • GitHub Actions

AUTHOR & EDITORIAL BOARD

Nexus Digital Systems
Architecture & Engineering Council

PUBLICATION & EDITION

October 2026 • v1.0 Enterprise
San Francisco & Istanbul

Preface & Executive Overview

This playbook documents the complete engineering, architectural, product, and operational lifecycle of building an enterprise-grade multi-tenant B2B SaaS platform from scratch within a modern software house (Nexus Digital Systems).

Departing from typical isolated coding manuals, this playbook explores the dynamic living interactions between every participant: from a Business Analyst conducting stakeholder interviews, to a CTO establishing cloud governance and error budgets; from a Principal Architect drawing Domain-Driven Design (DDD) Bounded Context boundaries, to a Junior Engineer mastering their first Value Object Pull Request; and from an SRE configuring Kubernetes clusters, to L1/L2 Technical Support engineers diagnosing production telemetry during live Sev-1 incidents.

Mandatory Technology & Engineering Stack

- **Backend Services:** Java 21, Spring Boot 3.3.x, Spring Cloud Gateway, Resilience4j Circuit Breakers.
- **Relational Data & Storage:** PostgreSQL 16 (Row-Level Security, dynamic schema provisioning, Flyway migrations).
- **Caching Layer:** Redis 7 (Google Cloud Memorystore L2 Cache-Aside & Token Bucket rate limiting).
- **Search & Forensics:** Elasticsearch 8.x (Real-time tenant audit trail and full-text log forensics).
- **Event Streaming:** Apache Kafka (Partitioning by TenantId for strict in-order stream processing).
- **Frontend Application:** Next.js 15 (App Router, React Server Components, TypeScript, Tailwind CSS).
- **Cloud & Orchestration:** Google Cloud Platform (GCP), Google Kubernetes Engine (GKE), Google Cloud Armor.
- **CI/CD Automation:** Multi-stage GitHub Actions pipeline enforcing automated test pyramids and security gates.

Table of Contents

1 SaaS Scenario, Company Vision, and Business Model 4
1.1 The Nexus Digital Systems Narrative 4
1.2 NexusCloud SaaS Product Architecture 4
1.3 Business Model, Pricing Tiers, and Contractual SLAs 4
1.4 Multi-Tenancy and Data Isolation Strategy 5
2 Organization Structure, Teams, and Roles 6
2.1 Team Topologies and Cross-Functional Squads 6
2.2 Detailed Role Profiles and Daily Cadences 6
3 Domain-Driven Design (DDD) Strategic Design & Event Storming 8
3.1 Problem Space vs. Solution Space 8
3.2 Event Storming Workshop & Discovered Flow 8
3.3 Ubiquitous Language Glossary 8

3.4 Bounded Contexts and Context Mapping 8
4 Domain-Driven Design (DDD) Tactical Design & Architecture 10
4.1 Hexagonal Architecture (Ports & Adapters) 10
4.2 Value Objects 10
4.3 Aggregate Roots and Invariants 10
4.4 Dual-Write Resolution via Transactional Outbox Pattern 10
5 Technology Stack & Architectural Blueprint 11
5.1 Backend: Java 21, Spring Boot 3, and Spring Cloud 11
5.2 Database: PostgreSQL 16 (Row-Level Security & Flyway) 11
5.3 Caching: Redis 7 (Google Cloud Memorystore) 11
5.4 Search & Analytics: Elasticsearch 8.x 11
5.5 Event Streaming: Apache Kafka 11
5.6 Frontend: Next.js 15 11
5.7 Cloud Platform: Google Cloud Platform (GCP) & GKE 12
6 Agile Scrum Lifecycle and Sprint Execution 13
6.1 Sprint 0 & Working Agreements 13
6.2 Definition of Ready (DoR) and Definition of Done (DoD) 13
6.3 User Story with Gherkin BDD Acceptance Criteria 13
6.4 Two-Week Sprint Cadence & Scrum of Scrums 13
7 DevOps, GCP, Kubernetes, and CI/CD Pipeline Architecture 14
7.1 GKE Cluster Topology & Workload Identity 14
7.2 Multi-Stage GitHub Actions Pipeline 14
7.3 Observability & Error Budget Mathematics 14
8 Quality Assurance (QA), Test Pyramid, and Security Architecture 15
8.1 Test Pyramid Execution Strategy 15
8.2 Enterprise SecOps Standards 15
9 Technical Support (L1/L2), Incident Ops, and Crisis Management 16
9.1 Support Tiers and Escalation SLAs 16
9.2 Real Outage Walkthrough: INC-4819 (Kafka Consumer Outage) 16
10 The Junior Engineer's Field Guide and Career Acceleration 17
10.1 30-60-90 Day Onboarding Roadmap 17
10.2 Converting User Stories into Clean Code 17
10.3 Navigating Senior Code Reviews 17
10.4 The Engineering Career Ladder 17

1 SaaS Scenario, Company Vision, and Business Model

1.1 The Nexus Digital Systems Narrative

Nexus Digital Systems is an enterprise product studio and software house engineering resilient cloud systems. Observing the common failure modes of corporate digital transformations:

1. Enterprise business workflows (approvals, reconciliations, API orchestrations) remain fragmented and brittle.
2. Interservice distributed events are lost or duplicated during network partitions.
3. Building multi-tenant data isolation and subscription infrastructure incurs prohibitive overhead.

Leadership commissioned **NexusCloud SaaS**—an intelligent workflow and event orchestration platform designed to serve global B2B enterprises while acting as the internal "Golden Path" engineering reference architecture.

1.2 NexusCloud SaaS Product Architecture

NexusCloud automates mission-critical business workflows, orchestrates third-party REST/gRPC APIs, executes stateful event-driven rule engines, and manages usage-based subscription monetization.

Core Platform Capabilities:

- **Multi-Tenancy:** Thousands of independent enterprise tenants operating on a unified infrastructure with rigorous data, caching, and streaming isolation.
- **Distributed Workflow Engine:** Resilient state machines triggered via webhooks, REST, or cron schedules with built-in retry and compensation logic.
- **Subscription & Revenue Cloud:** Dynamic tiers (Starter, Professional, Enterprise) with metered usage, automated dunning, and strict SLA tracking.
- **High-Throughput Event Streaming:** Apache Kafka ombudsman ingesting hundreds of thousands of events per second with zero message loss.
- **Real-Time Audit & Forensics:** Elasticsearch cluster providing sub-50ms audit log indexing and forensic search per tenant.

1.3 Business Model, Pricing Tiers, and Contractual SLAs

For enterprise customers, availability and contractually bound SLAs are non-negotiable buying criteria.

Plan Tier	Monthly Price	SLA Guarantee	Max Allowed Downtime	Support Level
Starter	$49 / mo	99.50% Uptime	Max. 3 hrs 39 mins	Standard Email (24h SLA)
Professional	$199 / mo	99.90% Uptime	Max. 43 mins 48 secs	Priority Chat & Email (4h SLA)
Enterprise	$799+ / mo	**99.95% Uptime**	**Max. 21 mins 54 secs**	**24/7 Phone & Slack TAM (< 15 min)**

Service Level Objectives (SLOs) & Indicators (SLIs)

- **Availability SLI:** Successful HTTP requests (2xx/3xx) over total valid requests >= 99.95%.
- **Latency SLI:** 99th percentile (p99) API response time < 150 ms.
- **Disaster Recovery Targets:** Recovery Time Objective (RTO) < 15 mins, Recovery Point Objective (RPO) < 1 min.

1.4 Multi-Tenancy and Data Isolation Strategy

NexusCloud leverages a **Hybrid Multi-Tenancy Architecture**:

1. **PostgreSQL Plane:** Starter and Professional tenants share tables protected by PostgreSQL **Row-Level Security (RLS)**. Enterprise tenants receive dynamically provisioned dedicated PostgreSQL schemas (`tenant_{id}`).
2. **Redis Cache Plane:** Keys are strictly scoped: `nexus:tenant:{tenant_id}:{aggregate}:{id}`.
3. **Apache Kafka Plane:** `tenant_id` serves as the Kafka message key, routing all tenant events to identical partitions to guarantee strict ordering.

2 Organization Structure, Teams, and Roles

2.1 Team Topologies and Cross-Functional Squads

The engineering department is structured into 5 autonomous Stream-Aligned squads backed by a Platform/SRE team and 24/7 Support Operations:

1. **Squad Alpha (Identity & Multi-Tenancy):** Tenant lifecycle, SSO/SAML 2.0, and RBAC governance.
2. **Squad Beta (Billing & Monetization):** Plans, metered usage aggregation, Stripe payments, and automated dunning.
3. **Squad Gamma (Workflow Core Engine):** Distributed state machines, trigger dispatchers, and Kafka event streaming.
4. **Squad Delta (Customer Experience & Web):** Next.js 15 tenant portal, visual workflow canvas, and design system.
5. **Squad Epsilon (DevOps, SRE & Platform):** Multi-zone GKE clusters, Cloud SQL, Kafka infra, Terraform IaC, and GitHub Actions.
6. **Customer Success & Operations (L1 / L2):** 24/7 incident intake, log forensics, and L3 on-call escalation.

2.2 Detailed Role Profiles and Daily Cadences

Chief Technology Officer (CTO) (Executive Leadership)

- **Responsibilities:** Oversee technology vision, optimize cloud spend (FinOps), ensure 99.95% SLA and SOC 2 Type II compliance.
- **Daily Cadence:** Audits Prometheus and Cloud Billing dashboards; presides over the Architecture Guild; signs off on high-impact ADRs.

Principal Software Architect (Architecture & System Design)

- **Responsibilities:** Guard Domain-Driven Design strategic boundaries, govern inter-service contracts, and manage technical debt.
- **Daily Cadence:** Maintains Context Maps; validates Kafka schema evolution; guides Anti-Corruption Layer (ACL) implementations.

Engineering Manager (EM) (People & Process)

- **Responsibilities:** Foster engineering talent, conduct weekly 1-on-1s, balance sprint velocity with sustainability, and remove blockers.

Product Owner (PO) & Business Analyst (BA) (Product & Requirements)

- **Responsibilities:** Translate market feedback into backlogs; author User Stories with Gherkin BDD (`Given-When-Then`) criteria; facilitate Event Storming.

Scrum Master (SM) & Agile Coach (Agile Facilitation)

- **Responsibilities:** Facilitate Scrum ceremonies; monitor velocity and burn-down charts; resolve cross-squad dependencies in Scrum of Scrums.

Lead / Senior Software Engineer (Technical Leadership & Core Coding)

- **Responsibilities:** Design Rich Domain Aggregates; perform rigorous PR reviews; lead weekly Pair Programming sessions with junior engineers.

Junior Software Engineer (Hands-on Development & Accelerated Growth)

- **Responsibilities:** Implement immutable Value Objects; write TDD unit tests; participate in Daily Scrums; absorb senior code review feedback.

DevOps & Site Reliability Engineer (SRE) (Infrastructure & Observability)

- **Responsibilities:** Maintain GKE clusters, Terraform modules, GitHub Actions CI/CD workflows, and Prometheus/Grafana alerting.

L1 / L2 Technical Support Engineer (Operations & Forensics)

- **Responsibilities:** Field customer tickets; query Elasticsearch logs and OpenTelemetry traces; execute support runbooks; triage Sev-1 crises.

3 Domain-Driven Design (DDD) Strategic Design & Event Storming

3.1 Problem Space vs. Solution Space

The architecture separates domains into three strategic tiers:

- **Core Domain (Workflow Orchestration):** The primary intellectual property and market differentiator. Hand-crafted by senior engineers.
- **Supporting Subdomains (Billing & Multi-Tenancy):** Essential business operations custom-built to support SaaS monetization models.
- **Generic Subdomains (Identity, Notification, Audit):** Commodity capabilities solved using industry standards (Spring Security, SendGrid, ELK).

3.2 Event Storming Workshop & Discovered Flow

Sprint 0 hosted an intensive 3-day Event Storming workshop:

1. `TenantRegistered` → `TrialSubscriptionProvisioned` → `DefaultWorkflowCreated`
2. `WorkflowTriggerReceived` → `TenantQuotaChecked` → `WorkflowExecutionStarted`
3. `StepExecutedSuccessfully` → `WorkflowExecutionCompleted` → `ExecutionMetered`
4. `BillingCycleEnded` → `InvoiceGenerated` → `PaymentChargedViaStripe` → `SubscriptionRenewed`
5. **(Crisis Path)**: `PaymentFailed` → `GracePeriodStarted` → `DunningEmailSent` → `SubscriptionSuspended`

3.3 Ubiquitous Language Glossary

Term (EN)	Term (TR)	Context	Formal Specification & Invariant
Tenant	Kiracı	Identity	Isolated customer organization possessing distinct database and security boundaries.
Subscription	Abonelik	Billing	Contract granting workflow capabilities and execution limits for a discrete period.
Plan Tier	Plan Seviyesi	Billing	Predefined packaging (Starter, Professional, Enterprise) dictating SLA and pricing.
Workflow	İş Akışı	Workflow	Executable state machine comprised of triggers, steps, and conditional evaluators.
Execution	Yürütme	Workflow	A concrete runtime instance of a workflow tracked for metering and billing.
Outbox	Olay Yolu	Shared Kernel	Database table guaranteeing atomic local transaction and Kafka event streaming.

3.4 Bounded Contexts and Context Mapping

- **Customer / Supplier:** Identity Context is Upstream; Subscription Context is Downstream.

- **Anti-Corruption Layer (ACL):** The Workflow Core Engine translates incoming external billing models into localized internal `QuotaPolicy` entities.
- **Open Host Service (OHS):** Identity Context publishes versioned event contracts over Kafka topics using JSON Schema.

4 Domain-Driven Design (DDD) Tactical Design & Architecture

4.1 Hexagonal Architecture (Ports & Adapters)

NexusCloud microservices enforce strict layer boundaries:

- **Driving Ports:** Application services receiving external HTTP and Kafka events.
- **Core Domain:** Entities, Value Objects, Aggregate Roots, and Domain Events.
- **Driven Ports:** Repositories and external service interfaces (`ISubscriptionRepository`).
- **Secondary Adapters:** Spring Data JPA PostgreSQL, RedisTemplate, and KafkaTemplate implementations.

4.2 Value Objects

- **No Conceptual Identity:** Defined solely by their attributes.
- **Immutability:** Encapsulated in Java records or final classes.
- **Examples:** `TenantId` (validates DNS slugs/UUIDs), `Money` (enforces currency consistency), `SubscriptionPlan` (encapsulates tiers, quotas, and SLAs).

4.3 Aggregate Roots and Invariants

An Aggregate defines a strict transactional consistency boundary:

- `SubscriptionAggregate` Invariants:
  1. Cancelled (`CANCELLED`) subscriptions cannot be upgraded, downgraded, or renewed.
  2. A tenant may possess at most one active or trial subscription concurrently.
  3. The subscription billing end date (`periodEnd`) must strictly follow the start date (`periodStart`).

4.4 Dual-Write Resolution via Transactional Outbox Pattern

Writing to PostgreSQL while publishing to Kafka introduces split-brain risks during network partitions. NexusCloud implements the **Transactional Outbox Pattern**:

1. The domain aggregate state and the outbox event payload are committed inside the exact same local PostgreSQL ACID transaction.
2. A resilient background process (Debezium CDC or transactional poller) streams outbox records to Apache Kafka.
3. Guarantees 100% at-least-once message delivery with zero lost billing or provisioning events.

5 Technology Stack & Architectural Blueprint

5.1 Backend: Java 21, Spring Boot 3, and Spring Cloud

- **Java 21 Virtual Threads (Loom):** Manages tens of thousands of concurrent I/O connections without thread pool exhaustion.
- **Spring Cloud Gateway:** Centralized ingress handling tenant JWT authentication, routing, and IP/tenant rate limiting.
- **Resilience4j:** Circuit breakers isolate external billing webhooks and payment gateways.

5.2 Database: PostgreSQL 16 (Row-Level Security & Flyway)

PostgreSQL enforces engine-level tenant segregation:

```
ALTER TABLE subscriptions ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation_policy ON subscriptions
     FOR ALL
     USING (tenant_id = current_setting('app.current_tenant_id', true));
```

All database schemas are version-controlled via Flyway migration scripts.

5.3 Caching: Redis 7 (Google Cloud Memorystore)

- **Cache-Aside Pattern:** Active tenant profiles and limits cached with 1-hour TTL. Over 94% of read requests resolve in memory (< 2 ms).
- **Token Bucket Rate Limiting:** Protects API gateways against rogue scripts with HTTP 429 Too Many Requests responses.

5.4 Search & Analytics: Elasticsearch 8.x

All workflow execution traces, step outputs, and billing audit trails stream to Elasticsearch. Support engineers utilize Kibana to search across millions of logs in sub-50ms query windows.

5.5 Event Streaming: Apache Kafka

- **Topic Format:** `nexus.{domain}.{event-type}.v1`.
- **Tenant Partitioning:** Partition key set to `tenant_id` to guarantee strict in-order message delivery per tenant.
- **Dead-Letter Queue (DLQ):** Poisoned messages redirect to `.dlq` topics after 3 failed retries.

5.6 Frontend: Next.js 15

Next.js 15 App Router and React Server Components (RSC) power the multi-tenant portal via dynamic `app/[tenant]/dashboard` routing. Styled using Tailwind CSS to meet WCAG AA accessibility standards.

5.7 Cloud Platform: Google Cloud Platform (GCP) & GKE

- **GKE Clusters:** Multi-zone high-availability node pools across region us-central1.
- **Google Cloud Armor:** Enterprise WAF and DDoS mitigation.
- **GCP Secret Manager:** Zero-hardcoded credentials policy.

6 Agile Scrum Lifecycle and Sprint Execution

6.1 Sprint 0 & Working Agreements

- Deep work focus block every morning until 10:00 AM (no scheduled meetings).
- PR review turnaround SLA: maximum 4 business hours.
- Broken CI/CD builds represent the squad's shared number-one priority ("Stop the line").

6.2 Definition of Ready (DoR) and Definition of Done (DoD)

Definition of Ready (DoR)

- Business value and persona clearly documented.
- At least 2 Gherkin BDD criteria defined.
- Figma screen specs and variants linked.
- Story points estimated via Planning Poker.

Definition of Done (DoD)

- Domain test coverage >= 85%.
- Testcontainers Postgres & Kafka tests pass.
- SonarQube quality gate green.
- Approved by at least 2 senior engineers.
- Staging synthetic smoke tests pass.

6.3 User Story with Gherkin BDD Acceptance Criteria

```
Feature: Subscription Plan Upgrade
  Scenario: Active Starter tenant upgrades to Professional
    Given A tenant with ID "tenant-acme-corp" and status "ACTIVE"
    And Current plan tier is "STARTER"
    When The tenant requests an upgrade to "PROFESSIONAL"
    Then The subscription status remains "ACTIVE" and plan updates to "PROFESSIONAL"
    And A "PlanUpgradedEvent" is published to Kafka topic "nexus.billing.subscription-
events.v1"
    And The updated quota of 100,000 runs is immediately written to Redis cache
```

6.4 Two-Week Sprint Cadence & Scrum of Scrums

- **Monday:** Sprint Planning (Capacity planning, Sprint Goal commitment).
- **Daily (09:45 AM):** 15-minute standup covering accomplishments, plans, and blockers.
- **Friday (Day 5):** Backlog Refinement.
- **Friday (Day 10):** Sprint Review (Live demo in staging) & Retrospective (4L framework).
- **Tuesday & Thursday:** 20-minute cross-squad Scrum of Scrums (SoS).

7 DevOps, GCP, Kubernetes, and CI/CD Pipeline Architecture

7.1 GKE Cluster Topology & Workload Identity

- **Workload Identity Federation:** Kubernetes pods authenticate to GCP resources dynamically via OIDC without static JSON service account keys.
- **Horizontal Pod Autoscaler (HPA):** Scales pods from 3 to 15 when CPU exceeds 70% or memory exceeds 80%.
- **Zero-Downtime Rolling Updates:** Enforced with `maxSurge: 1` and `maxUnavailable: 0` alongside Actuator probes.

7.2 Multi-Stage GitHub Actions Pipeline

1. **Static Analysis:** Checkstyle, SonarQube Quality Gate, and Trivy filesystem vulnerability scans.
2. **Automated Testing:** Real-container integration tests using Testcontainers (PostgreSQL 16 & Kafka).
3. **Packaging:** Multi-stage Docker builds based on Alpine Temurin 21 with non-root security.
4. **Deployment:** Image push to Google Artifact Registry and rollout verification via `kubectl`.

7.3 Observability & Error Budget Mathematics

- **Prometheus & Grafana:** Monitoring API response times (p99 < 150 ms), connection pool states, and Kafka consumer lags.
- **Structured JSON Logging:** Logs include `traceId`, `spanId`, and `tenantId` metadata.
- **OpenTelemetry & Jaeger:** Distributed tracing across distributed asynchronous microservices.
- **Error Budget Governance:** A 99.95% SLA permits 21.6 minutes of monthly downtime. Consuming > 80% freezes new feature deployments in favor of reliability hardening.

8 Quality Assurance (QA), Test Pyramid, and Security Architecture

8.1 Test Pyramid Execution Strategy

- **Unit Tests (50%):** JUnit 5 and Mockito testing domain invariants in sub-second execution windows.
- **Integration Tests (30%):** Testcontainers verifying real SQL queries, RLS policies, and Kafka consumer offsets.
- **Contract Tests (15%):** Pact verifying API consumer-provider schema integrity.
- **End-to-End Tests (5%):** Playwright browser tests simulating end-to-end customer registration and payments.

8.2 Enterprise SecOps Standards

- **OWASP Top 10 Mitigation:** Parameterized JPA queries eliminate SQL injection risks.
- **Tenant Isolation Verification:** Penetration test suites confirm cross-tenant API requests trigger HTTP 403 Forbidden.
- **Zero-Hardcoded Secrets:** Credentials retrieved dynamically at runtime from GCP Secret Manager.

9 Technical Support (L1/L2), Incident Ops, and Crisis Management

9.1 Support Tiers and Escalation SLAs

- **L1 Support (Helpdesk):** 15-minute response SLA. Fields tickets, executes standard runbooks, and classifies priority.
- **L2 Technical Support:** 30-minute SLA. Analyzes Elasticsearch logs and reproduces issues in staging.
- **L3 Squad On-Call & SRE:** 15-minute Sev-1 SLA. Joins active War Rooms to deploy hotfixes and mitigate outages.

9.2 Real Outage Walkthrough: INC-4819 (Kafka Consumer Outage)

- **Incident:** 12,000 burst webhooks caused container OOM failures and a 15,000-message Kafka lag surge.
- **Mitigation:** Incident Commander opened a War Room; memory limits scaled to 2048Mi; replicas expanded to 10. Normal operations restored within 28 minutes.
- **5 Whys Analysis:** Revealed HPA lacked queue-lag visibility. KEDA event-driven autoscaling was deployed cluster-wide.
- **Blameless Post-Mortem:** Focus shifted to systemic resilience rather than individual blame.

10 The Junior Engineer's Field Guide and Career Acceleration

10.1 30-60-90 Day Onboarding Roadmap

- **Days 1 - 30:** Spin up local environment via Docker Compose, master Ubiquitous Language, deliver first Value Object PR.
- **Days 31 - 60:** Deliver a complete User Story independently, write Testcontainers integration tests, attend bi-weekly pairing sessions.
- **Days 61 - 90:** Implement a Kafka consumer or Redis caching strategy, shadow on-call rotations during Sev-1 events.

10.2 Converting User Stories into Clean Code

1. Clarify acceptance criteria (`Given-When-Then`) with the PO during refinement.
2. Apply Test-Driven Development (TDD) by writing the failing JUnit test first (Red).
3. Write the minimal clean implementation to pass the test (Green).
4. Refactor and adhere strictly to DDD layer boundaries.
5. Submit atomic PRs (< 250 lines changed).

10.3 Navigating Senior Code Reviews

- Senior feedback is a technical investment in your craft, never a personal critique.
- Approach feedback with curiosity: **"Could you explain the architectural advantages of this approach?"**
- The 15-Minute Rule: Investigate logs, traces, and code for 15 minutes before escalating questions with structured context.

10.4 The Engineering Career Ladder

- **Junior:** Writes well-tested code with guidance; actively learns domain standards.
- **Mid-Level:** Delivers complex features independently; optimizes database interactions.
- **Senior / Lead:** Architects rich aggregate roots; mentors team members; conducts thorough reviews.
- **Principal Architect / CTO:** Sets company-wide engineering vision and distributed standards.